Comandos fundamentais do Python

Site: Ambiente Virtual de Aprendizagem - UFR
Course: ALGORITMOS E PROGRAMAÇÃO
Book: Comandos fundamentais do Python
Printed by: Nabor do Nascimento da Silva
Date: Sunday, 5 April 2026, 4:43 PM

Table of contents

1. Introdução

2. Comentários
2.1. Cabeçalho do programa

3. Execução de linhas de comando

4. Variável
4.1. Tipos de dados
4.2. Tipos de dados em Python
4.3. Declaração de variáveis
4.4. Declaração de variáveis em Python
4.5. Regras para a criação de variáveis
4.6. Palavras reservadas

5. Atribuição de valores

6. Operadores matemáticos
6.1. Linearização das operações
6.2. Precedência entre operadores aritméticos
6.3. Funções matemáticas

7. Entrada de dados pelo teclado
7.1. Entrada de dados - simples
7.2. Entrada de dados somente com input

8. Saída de dados
8.1. Escapes

9. Estruturas de decisão
9.1. Operadores de teste
9.2. Estrutura de Decisão SIMPLES
9.3. Estrutura de Decisão COMPOSTA
9.4. Estruturas de decisão encadeadas
9.5. Estrutura Múltipla Escolha
9.6. Comparação entre as estruturas de decisao

1. Introdução

Python é uma linguagem de programação de alto nível, interpretada de script, imperativa, orientada a objetos, funcional, de tipagem dinâmica e forte. Foi lançada por Guido van Rossum em 1991.

O nome Python teve a sua origem no grupo humorístico britânico Monty Python, criador do programa Monty Python's Flying Circus, embora muitas pessoas façam associação com o réptil do mesmo nome (em português, píton ou pitão).

Python é uma linguagem de propósito geral multiparadigma, suporta o paradigma orientado a objetos, imperativo, funcional e procedural. Possui tipagem dinâmica e uma de suas principais características é permitir a fácil leitura do código e exigir poucas linhas de código se comparado ao mesmo programa em outras linguagens.

Devido às suas características, ela é utilizada, principalmente, para processamento de textos, dados científicos e criação de CGIs para páginas dinâmicas para a web.

Para criar programas em Python precisamo de um interpretador.

É possível baixar e instalar um interpretador Python no endereço:

- https://www.python.org/downloads/

Opcionalmente, como inicio de estudos na linguagem, podemos executar código em Python em um interpretador online, que pode ser acessado em um navegador de internet, como por exemplo o interpretador no endereço:

- https://www.online-python.com/

Existen diversos tutorias de apoio na internet. A seguir apresento uma sugestão:

- https://www.w3schools.com/python/default.asp

2. Comentários

O que são comentários?

São linhas de textos que cuja marcação inicial e final indicam ao Python que o texto não contém comandos para serem processados.

São usados para adicionar explicações ou observações ao código-fonte. Servem principalmente para documentar trechos do programa para facilitar a compreensão de seu funcionamento caso seja necessario realizar alguma tipo de manutenção no futuro.
Em Pythhon é possível efetuar comentários de duas formas:

a) comentário como uma linha apenas

Nesse caso utilizamos o caractere cerquilha *#, assim* tudo que estiver vier depois desse caractere, deve ser ignorado pelo interpretador do Python. Por exemplo:

```
#tudo que estiver a frente da cerquilha será ignorado na execução
```

Um comentário pode aparecer no inicio da linha ou após espaço em branco ou código, mas não dentro de uma string literal. O caracterer cerquilha dentro de uma string literal é apenas uma cerquilha.

b) comentário na forma de um conjunto de linhas

Essa notação se dá pelo uso de **3 aspas simples, ou entao, 3 aspas duplas**. Nestes casos, tudo que estiver delimitado pelas aspas não será interpretado, por exemplo:

```
'''
toda informações contida entre 3 aspas SIMPLES
é considerada como caracteres que devem ser ignorados na execução do programa.
'''
```

2.1. Cabeçalho do programa

Como prática de desenvolvimento profissional, o cabeçalho deve ter dados fundamentais para a sua compreensão. Eles podem indicar:

- identificação do algoritmo (código, número, nome);
- data da criação;
- data da última alteração;
- versão;
- Objetivos ou funções.

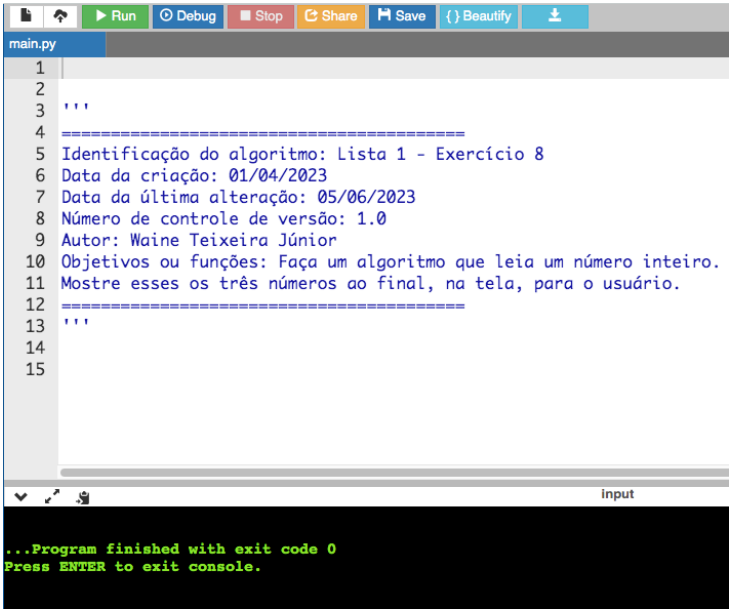
Por exemplo:

```
'''
=========================================
Identificação do algoritmo: Lista 1 - Exercício 8
Data da criação: 01/04/2023
Data da última alteração: 05/06/2023
Número de controle de versão: 1.0
Autor: Waine Teixeira Júnior
Objetivos ou funções: Faça um algoritmo que leia um número inteiro. Depois disso, calcule o sucessor e o antecessor do número lido.
Mostre esses os três números ao final, na tela, para o usuário.
=========================================
'''
```

Ao executar o programa com o botão RUN, o Python vai apresentar uma mensagem de retorno do valor zero como saída de execução:

- Programa finished with exit code 0.

Essa mensagem indica que o programa foi executado e finalizado.



3. Execução de linhas de comando

O Python executa programas com um interpretador.

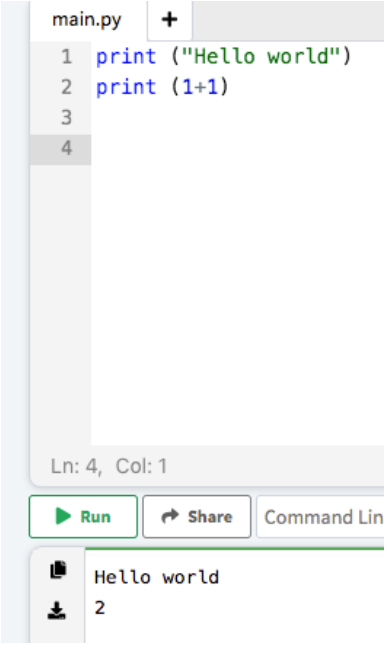
Isso significa que os comandos digitados são executados sequencialmente e o resultado é mostrado diretamente na tela de saída de execução.

Experimente realizar:

print ("Hello world")

print (1+1)

Execução:



4. Variável

O que é uma variável?

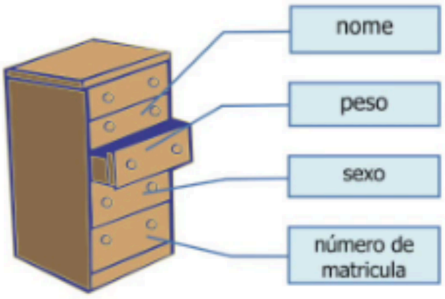
Uma variável de memória é um local na memória RAM de um computador onde é armazenado um valor.

Em linguagem de programação, uma variável é usada para representar esse local na memória e permitir acesso e manipulação dos dados armazenados.

O nome da variável é usado como uma referência a esse endereço de memória, permitindo que o valor armazenado seja recuperado ou alterado ao longo do tempo.

No exemplo a seguir, é apresentado uma analogia de memória do computador como se fosse um armário. Nessa analogia, as gavetas são as variáveis, pois cada gaveta guarda uma variável diferente: as gavetas ou variáveis utilizadas são: nome, peso, sexo e número de matrícula

Importante: quando criamos uma variável simples, ela pode armazenar apenas um dado. Por exemplo, no caso abaixo, nome poderia armazenar o dado Maria da Silva.



Obs: O que é memória RAM?

RAM (Random Access Memory) é a principal memória de um computador, responsável por armazenar dados temporários enquanto o computador está em funcionamento.
É chamada de "acesso aleatório" porque o sistema operacional e os programas podem acessar qualquer posição de memória RAM a qualquer momento, sem precisar ler os dados consecutivamente como acontece com outros tipos de memória, como discos rígidos ou unidades de fita.
A memória RAM é geralmente limpa quando o computador é desligado, tornando necessário o armazenamento de dados permanentes em outro tipo de memória, como o disco rígido ou unidade de estado sólido.

4.1. Tipos de dados

Todas as linguagens de programação utilizam um tipo de dado associado a uma variável.

Quando uma variável é declarada, o computador reserva um espaço na memória para armazenar seu valor.

Cada tipo de variável em uma linguagem de programação tem uma quantidade fixa de memória associada. De certa forma, é como se tivéssemos compartimentos específicos para cada tipo de dado que será armazenado. No exemplo a seguir, um guarda-roupas possui compartimentos que são específicos para determinados tipos de vestuário e roupa de cama por exemplo, calceira, sapateira, maleiro, prateleiras e nichos.

Em outras palavras, em linguagem de programação, um tipo de variável é uma categoria que define o tipo de dado que uma variável pode armazenar.

O programador precisa conhecer o tipos de dados disponíveis para criar as estruturas corretas que irão armazenados os dados que o programa irá processar.

Os tipos de dados básicos, também chamados de primitivos são:

Nome do tipo em portugol	Tipo do dado que armazena	Exemplo de nome de variável	Exemplo de dado armazenado	
Inteiro	números inteiros, não aceita valores após a vírgula	numeroDeFilhos	3	
Real, ponto flutuante ou decimal	números com valores após a vírgula	pi	3,141592653	
Cadeia de caracteres (string)	conjunto de caracteres	nomeDoCliente	Paulo Araújo	
Lógico (boolean)	Lógico	status	verdadeiro	

A partir dos tipos de dados primitivos, outros tipos derivados foram criados. Trata-se de uma subdivisão de um tipo primitivo.

A linguagem C por exemplo, possui os seguintes tipos e subtipos:

Sequência	Bytes ocupados	Faixa (ou intervalo)
short int signed int signed short int wchat_t signed wchar_t	2	-32.768 a 32.767
unsigned short int unsigned wchar_t	2	0 a 65.535
int long int signed long int	4	-2.147.483.648 até 2.147.483.647
unsigned int unsigned long int	4	0 a 4.294.967.295
char signed char	1	-128 a 127
unsigned char	1	0 a 255
bool	1	*true* ou *false*
float	4	1,2e-38 a 3,4e+38
double long float	8	2,2e-308 a 1,8e+308

O tipo de variável determina a quantidade de memória que será alocada para armazenar o valor da variável, bem como as operações que podem ser realizadas com essa variável. Na tabela acima podemos ver os valores a serem ocupados por cada dado armazenado no tipo correto. Os valores estão dispostos na coluna Bytes ocupados.

Essa é a principal razão para o uso de tipos. Ao determinar o tipo, determinamos a quantidade de memória que será ocupada por cada dado armazenado. Individualmente não representa muita coisa. Mas se considerarmos que grandes bases de dados podem possuir milhares de dados em um campo, como é o caso da Receita Federal e a base do IRPF por exemplo, isso faz muito sentido, em termos de otimização do uso de memória.

A combinação de variável e seu tipo correto é uma forma de planejar o uso otimizado da memória.

Muito embora hoje tenhamos desenvolvido a tecnologia de armazenamento, ela ainda é cara e limitada.

4.2. Tipos de dados em Python

Python é uma linguagem dinamicamente tipada, o que significa que não é necessário declarar o tipo de variável ou fazer casting (mudar o tipo de variável), pois o Interpretador se encarrega disso para nós. Entretanto, podemos também indicar o tipo desejado para uma variável.

Os principais tipos de dados padrão do Python são:

Nome do tipo em portugol	Nome do tipo em Python	Tipo do dado que armazena	Exemplo de nome de variável	Exemplo de dado armazenado
Inteiro	int	números inteiros, não aceita valores após a vírgula	numeroDeFilhos	3
Real, ponto flutuante ou decimal	float	números com valores após a vírgula	pi	3.141592653
Cadeia de caracteres (string)	str	conjunto de caracteres	nomeDoCliente	Paulo Araújo
Lógico (boolean)	bool	Lógico	status	verdadeiro

a) Tipo Inteiro (int)

O tipo inteiro é um tipo composto por caracteres numéricos (algarismos) inteiros. É um tipo usado para um número que pode ser escrito sem um componente decimal, podendo ter ou não sinal, isto é: ser positivo ou negativo. Por exemplo, 21, 4, 0, e −2048 são números inteiros, enquanto 9.75, 1/2, 1.5 não são.

b) Ponto Flutuante ou Decimal (float)

É um tipo composto por caracteres numéricos (algarismo) decimais. O famoso ponto flutuante é um tipo usado para números racionais (números que podem ser representados por uma fração) informalmente conhecido como “número quebrado”.

c) String (str)

É um conjunto de caracteres dispostos numa determinada ordem, geralmente utilizada para representar palavras, frases ou textos.

d) Boolean (bool)

Tipo de dado lógico que pode assumir apenas dois valores: falso ou verdadeiro (False ou True em Python). Na lógica computacional, podem ser considerados como 0 ou 1, true ou false, verdadeiro ou falso.

Quando fazemos atribuições no programa, o Python vai atribuir o valor a um tipo automaticamente, de acordo com o conteúdo do valor.

É possível verificar o tipo com o comando

```
• print (type(nomeDaVariavel)).
```

Exemplos:

Figura - Tipos de dados atribuídos em Python

main.py

```python
#exemplos de atribuições de valores
idade = 18
ano = 2002
altura = 1.80
peso = 73.55
nome = 'Guilherme'
profissao = 'Engenheiro de Software'
fimDeSemana = True
feriado = False
#verificando os tipos de dados que o Python atribuiu
print('tipo para idade = ',type(idade))
print('tipo para ano = ',type(ano))
print('tipo para peso = ',type(peso))
print('tipo para altura = ',type(altura))
print('tipo para profissao = ',type(profissao))
print('tipo para nome = ',type(nome))
print('tipo para fimDeSemana = ',type(fimDeSemana))
print('tipo para feriado = ',type(feriado))
```

Ln: 17, Col: 45

Run Share Command Line Arguments

```
tipo para ano =  <class 'int'>
tipo para peso =  <class 'float'>
tipo para altura =  <class 'float'>
tipo para profissao =  <class 'str'>
tipo para nome =  <class 'str'>
tipo para fimDeSemana =  <class 'bool'>
tipo para feriado =  <class 'bool'>


** Process exited - Return Code: 0 **
Press Enter to exit terminal
```

4.3. Declaração de variáveis

Quando as linguagens de programação foram inventadas, os desenvolvedoes determinaram que os tipos de dados deveriam ser definidos no início do programa.

Isso acontece na maior parte das linguagens de programação mais conhecidas.

Por exemplo:

C	PASCAL
• Não precisa de iniciar com var • Inicia com o tipo, depois o nome da variável e finalizar com ;	• Precisa iniciar com var • Inicia o nome da variável, o sinal de :, o tipo e finalizar com ;
	var
int idade;	idade: integer;
char tipoSangue;	tipoSangue: char;
char doadorOrgaos; obs: C não possui o tipo lógico (boolean)	doadorOrgaos: boolean;
float altura;	altura: real;

Porque declaramos variáveis no início do programa?

Nas linguagens de programação que utilizam compiladores, tais como C e Pascal, as etapas de compilação fazem diversas verificações: sintática, semântica e léxica. Qualquer dessas etapas que apresentar problema vai impedir que o programa seja compilado. Se não houver problemas, quando o código for execupado, a primeira coisa que o sistema operacional fará será verificar quais variáveis foram declaradas e seus tipos, e verificar se há espaço suficiente de memória para executar o programa que irá utilizá-las.

4.4. Declaração de variáveis em Python

O Python possui tipagem dinâmica. Isso quer dizer que as variáveis e seus tipos podem ser criados a qualquer momento, não necessitando obrigatoriamente estar declarados em uma sessão, como é o caso de outras linguagens com tipagem estática, como é o caso de Pascal ou C.

Entretanto, declarar variáveis, mesmo que em comentários, ajuda na manutenção do código futuramente. Dessa forma, recomendamos que a declaração seja feita na forma de comentários, no início do programa, logo após a seção de cabeçalho.

Por exemplo:

```
'''
# ========================================
Variáveis
Nome: tipo;
# ========================================
'''
```

4.5. Regras para a criação de variáveis

Obervar as seguintes regras para o uso de variáveis :

- Variáveis podem conter tanto letras quanto números, mas têm de começar obrigatoriamente com uma letra;
- A diferenciação do nome da variável utilizando maiúsculas e minúsculas muda de acordo com a linguagem utilizada. Python segue a diferenciação. portanto uma variável A e outra a são diferentes;
- Não utilizar acentuação gráfica;
- Não utilizar caracteres especiais;
- Não iniciar nomes de variáveis com números;
- Toda linguagem tem um conjunto de palavras reservadas, que devem ser consultada quando iniciar seus estudos na linguagem.
- Nomes de variáveis devem ser representativos. Não utilize nomenclaturas com abreviações que dificultam a compreensão, como por exemplo x4rt20, a não ser que seja um nome representativo para o domínio da aplicação ou programa;
- Por padronização, utilizamos a inicial minúscula. Se houver nomes compostos, não utilizar *underline* nem traço. Juntar os nomes iniciando a segunda palavra com letra maiúscula. Exemplo: nomeDaVariavel;
- Se você der a uma variável um nome inválido para uma variável, isso causará um erro de sintaxe: Exemplo: 76trombones = "grande parada" vai causar o erro: SyntaxError: invalid syntax

Exemplo:

```
# ========================================
'''
Seção de Declarações de variáveis
Nome: str
salario: float
anoNascimento: int
'''
# ========================================
```

Observações:

- O tipo str equivale ao tipo caractere no pseudocódigo;
- O tipo float equivale ao tipo real em pseudocódigo;
- O tipo int equivale ao tipo inteiro em pseudocódigo.

Trataremos mais sobre os tipos de dados na seção atribuição.

4.6. Palavras reservadas

Não podemos usar essas palavras reservadas como variáveis, labels ou nomes de função:

and	as	assert	break
class	continue	def	del
elif	else	except	False
finally	for	from	global
if	import	in	is
lambda	None	nonlocal	not
or	pass	raise	return
True	try	while	with
yield			

5. Atribuição de valores

A Atribuição de Valores é a passagem de informação a determinada variável. Toda variável, por sua definição, pode receber valores ou então, pode ter seu valor alterado, como estudado anteriormente nas aulas Variáveis em Python e Características das Variáveis em Python . Nesse primeiro momento, estudaremos a atribuição simples, isto é, a forma comum de atribuição.

A linguagem Python tem definido que o sinal que conhecemos pelo nome de igual (=) será o operador de atribuição. Desta forma, iremos construir expressões compostas de 2 partes, ou melhor que estarão em um dos dois lados do operador de atribuição: do lado esquerdo e do lado direito.

A parte ao lado esquerdo do operador de atribuição sempre receberá o valor definido no lado direito do operador.

Assim, é uma norma da linguagem que o lado esquerdo possua uma referência que possa ter seu valor alterado, enquanto que do lado direito, haja algum valor a ser atribuido à referência ao lado esquerdo.

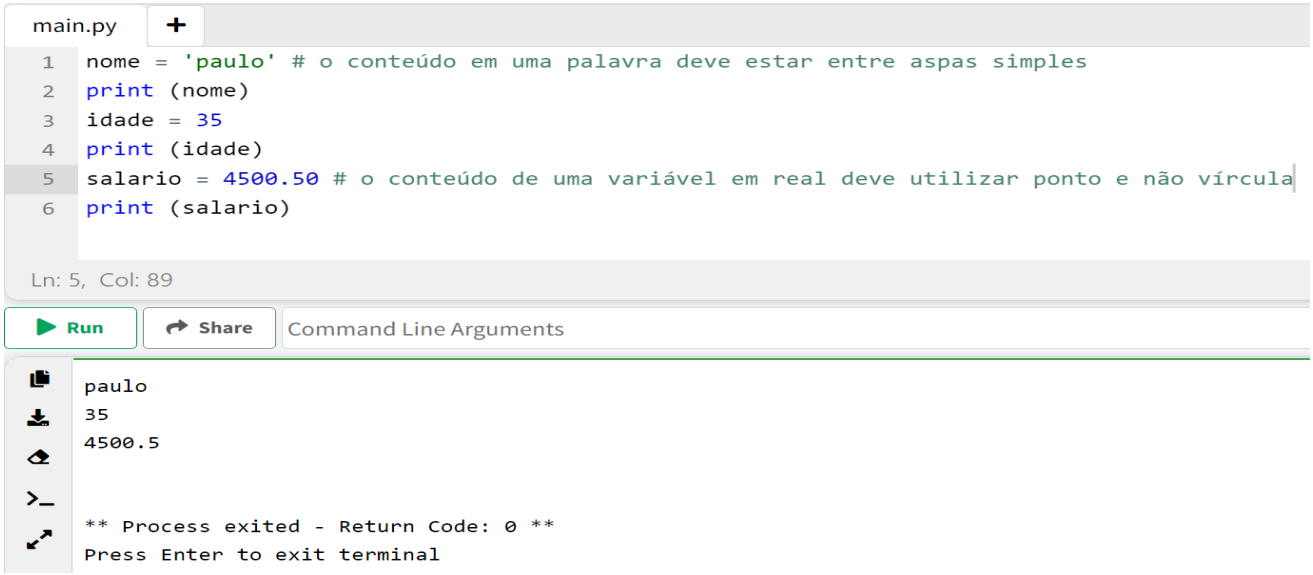
<variável> = <valor>

Observação: valores alfanuméricos precisam estar entre aspas simples ou duplas.

Exemplos:

- nome = 'Paulo'
- idade = 35
- salario = 4500.50

Figura - Exemplo de atribuição de variáveis em Python



6. Operadores matemáticos

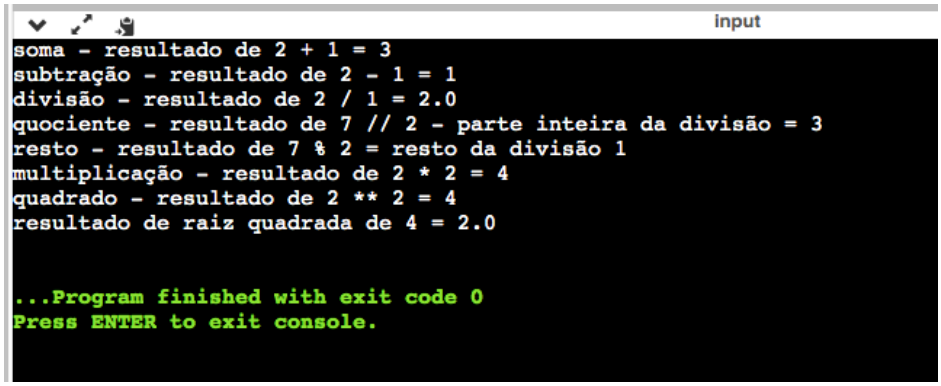
Os operadores matemáticos em Python são:

Operador	Comando	Exemplo
Soma	+	print (2+1)
Subtração	-	print (2-1)
Divisão	/	print (2/1)
Quociente da divisão (parte inteira) - equivale ao DIV em Portugol	//	print (2//1)
Resto da divisão	%	print (2%1)
Multiplicação	*	print (2*2)
Potenciação	**	print (2**2)
Raiz	Raizq (expressão)	import math print(math.sqrt(4))

Exemplos em Python

```
print ('soma - resultado de 2 + 1 =', 2 + 1)
print ('subtração - resultado de 2 - 1 =', 2 - 1)
print ('divisão - resultado de 2 / 1 =', 2 / 1)
print ('quociente - resultado de 7 // 2 - parte inteira da divisão =', 7 // 2)
print ('resto - resultado de 7 % 2 = resto da divisão', 7 % 2)
print ('multiplicação - resultado de 2 * 2 =', 2 * 2)
print ('quadrado - resultado de 2 ** 2 =', 2 ** 2)
# math.sqrt é uma função matemática que necessida do comando IMPORT MATH
import math
print('resultado de raiz quadrada de 4 =',math.sqrt(4))
```

Figura - Exemplos em Python



6.1. Linearização das operações

Para a construção de algoritmos que realizam cálculo matemáticos, todas as expressões aritméticas devem ser linearizadas, ou seja, colocadas em linhas, devendo também ser feito o mapeamento dos operadores da aritmética tradicional para os do Portugol.

O Portugol utiliza apenas parênteses para representar as expressões matemáticas. Portanto, é preciso indicar com camadas de parêntes para substuir os dinais de chaves e colhetes.

Por exemplo:

Tradicional	Computacional
$\left\{\left[\frac{2}{3}-(5-3)\right]+1\right\}.5$	((2/3-(5-3))+1)*5

Como resultado da linearização temos a fórmula:

- ((2/3-(5-3))+1))*5

Supondo que o número 3 fosse uma variável da equação, como por exemplo, y, poderiamos transcrever a fóruma da seguinte forma:

- x = ((2/y-(5-y))+1))*5

Essa equação também poderia ser escrita em termos de uma função de x, por exemplo:

```
• F(x) = ((2/x-(5-x))+1))*5
```

Note que o nome da variável foi Fx e não F(x) porque não podemos utilizar caracteres especiais, como parênteses, cerquilha, etc. na composição de um nome de variável.

Exemplo de aplicação:

Faça um algoritmo que receba o valor de x e calcule a função: F(x) = ((2/x-(5-x))+1))*5. O valor de x deve ser informado pelo usuário do programa.

6.2. Precedência entre operadores aritméticos

As linguagens de programação estabelecem a prioriade de operações aritméticas assim como na aritmética tradicional, da seguinte forma:

- 1º Parênteses internos
- 2º Potenciação e raiz
- 3º Multiplicação, divisão, função para retornar o divisor e função para retornar o resto
- 4º Soma e subtração

6.3. Funções matemáticas

Principais funções matemáticas do Python:

O Python oferece uma ampla variedade de funções matemáticas integradas por meio do módulo math.

Para utilizar essas funções, você precisa importar o módulo math em seu programa.

- **Import math**

Aqui estão algumas das principais funções matemáticas disponíveis em Python:

Funções de Potência:

- math.pow(x, y): Retorna x elevado à potência y.
- math.sqrt(x): Retorna a raiz quadrada de x.

Funções de Arredondamento:

- math.ceil(x): Retorna o menor número inteiro maior ou igual a x (teto).
- math.floor(x): Retorna o maior número inteiro menor ou igual a x (piso).
- math.trunc(x): Retorna a parte inteira de x (sem arredondamento).

Constantes Matemáticas:

- math.pi: Valor de π (pi).
- math.e: Valor da constante matemática "e".

Funções Trigonométricas:

- math.sin(x): Retorna o seno do ângulo x em radianos.
- math.cos(x): Retorna o cosseno do ângulo x em radianos.
- math.tan(x): Retorna a tangente do ângulo x em radianos.
- math.asin(x): Retorna o arco seno de x em radianos.
- math.acos(x): Retorna o arco cosseno de x em radianos.
- math.atan(x): Retorna o arco tangente de x em radianos.

Funções Exponenciais e Logarítmicas:

- math.exp(x): Retorna e elevado à potência x.
- math.log(x): Retorna o logaritmo natural de x.
- math.log10(x): Retorna o logaritmo base 10 de x.

Funções de Conversão Angular:

- math.degrees(x): Converte ângulos de radianos para graus.
- math.radians(x): Converte ângulos de graus para radianos.

7. Entrada de dados pelo teclado

Existem 3 tipos formas de entrada de dados pelo teclado, a saber.

7.1. Entrada de dados - simples

A forma mais simples de entrada de dados em Python é realizada com o comando INPUT ().

O comando input 👇 significa ENTRADA e é equivalente ao comando leia 👇 .

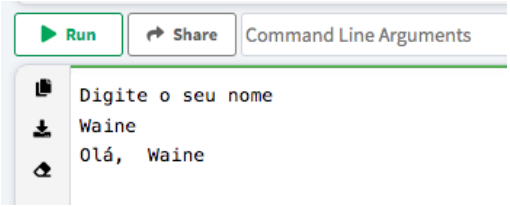
Para realizarmos a leitura de uma variável, é preciso inicialmente informar a variável que receberá o valor que será lido pelo teclado, da seguinte forma:

nomeDaVariavel = input ()

Exemplo:

```
print ("Digite o seu nome")
nome = input ()
print ("Olá, ", nome)
```

Figura - Execução do programa em Python



Observação:

O comando `input()` em Python lê uma linha da entrada padrão (geralmente do teclado) e retorna essa linha como uma string. Portanto, quando você utiliza o comando `variavel = input()`, o valor digitado pelo usuário é armazenado na variável como uma string.

Por exemplo:

```
print ("Digite seu nome: ")
nome = input()
print("Olá, " + nome + "!")
```

Portanto, para armazenar valores de outros tipos, atribua o tipo correto na leitura.

Por exemplo:

```
print ("Digite sua idade: ")
idade = int (input())
print("Idade = ", idade)
```

7.2. Entrada de dados somente com input

a) Entrada de dados com mensagem no comando INPUT

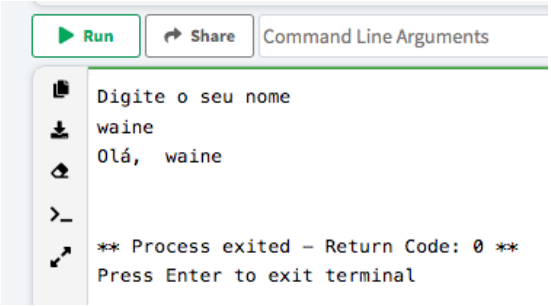
O comando INPUT () permite que seja impressa uma mensagem na tela dentro dos parênteses, dispensando o uso do PRINT para indicar a leitura dos dados, da seguinte forma:

nomeDaVariável = input ("mensagem")

Exemplo:

```
nome = input ("Digite o seu nome")
print ("Olá, ", nome)
```

Figura - Execução do programa em Python



8. Saída de dados

O comando que executa saída de dados é o comando **escreva em pseudocódigo**. Em Python utilizamos o comando print.

```
• print( )
```

O comando print pode ser utilizado em quatro situações:

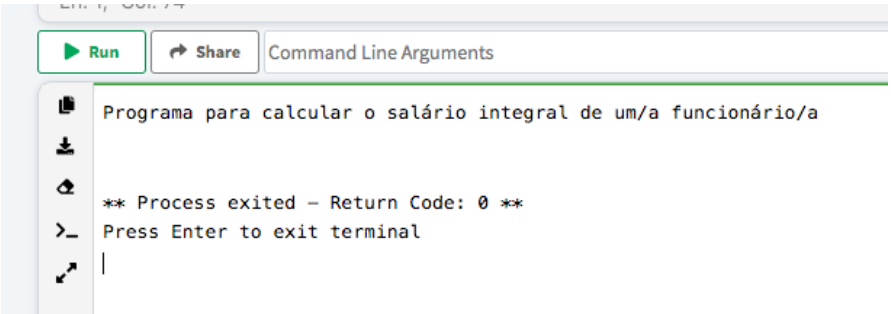
a) Mostrar uma mensagem na tela, da forma que o projetista definiu.

Observe que nesse caso, o uso de aspas (**simples ou duplas)** é obrigatório. Exemplo:

```
• print ('Programa para calcular o salário integral de um/a funcionário/a')
```

A tela de console apresentará o seguinte resultado:

Figura - Resultado do comando print



b) Mostrar o conteúdo de uma variável que está na memória do computador. Nesse caso NÃO utilizamos as aspas.

Exemplos:

```
• print (nome)
• print (idade)
• print (salario)
```

c) Combinar a saída de uma mensagem e valor de uma variável

Podemos combinar texto e conteúdos de variáveis, desde que sigam a regra mássica, aspas duplas para texto pré-determinado e sem aspas para mostrar valor de variável. Entretanto, devem-se separar variáveis e mensagens de tela com vírgula.

Exemplo:

- print ('Salário = ', salario)

d) Mostrar o resultado de uma operação matemática
Embora não seja recomendado o uso dessa forma, deve-se saber que é possível mostrar o resultado de uma operação matemática sem a necessidade de atribuição a variável. Exemplo:

```
• print (5+5);
```

e) Mostrar um valor de tipo flot formatado adequadamente

a formatação de saída de dados com f-string em Python permite que você controle como os valores numéricos são apresentados na impressão do texto. O f-string é uma forma mais conveniente e legível de formatar strings incorporando variáveis e expressões dentro delas.

Exemplos:

- print(f"Valor do produto: R$ {valorProduto:.2f}")
- print(f"Valor do produto com desconto: R$ {valorComDesconto:.2f}")

As chaves {} dentro da string delimitam o local onde as variáveis ou expressões serão inseridas.

A parte antes do : dentro das chaves contém o nome da variável que será inserida na string, neste caso, valorProduto e valorComDesconto.

O :.2f após o : especifica a formatação do valor numérico. Neste caso, o .2f significa que queremos exibir o valor com duas casas decimais (arredondadas, se necessário) para valores de ponto flutuante (float).

Recomendação: evitar utilizar o comando de saída de dados para calcular operações

Veremos em situações futuras que dificilmente fazemos cálculos pequenos e simples assim. Trabalharemos com situações nas quais o resultado de um cálculo tem grandes chances de ser referenciado e necessário para outros cálculos em outras partes do programa, de forma que, se o cálculo não for armazenado em uma variável, ele terá que ser refeito todas as vezes que seja necessário.

Muitos problemas decorrem da não escolha por variável. Aumento da possibilidade de erros, dificuldade de rastreamento de erros e diminuição do desempenho do programa são os mais comuns.

8.1. Escapes

A formatação de escapes em Python refere-se ao uso de caracteres especiais, chamados "caracteres de escape", para representar caracteres não imprimíveis ou para realizar a formatação específica na saída de dados. Esses caracteres são precedidos por uma barra invertida \.

Alguns dos caracteres de escape comuns incluem:

Sequência	Descrição
\n	Insere uma quebra de linha.
\t	Insere tabulação horizontal.
\v	Insere tabulação vertical.
\r	Equivalente ao efeito da tecla Enter.
\'	Aspas simples.
\"	Aspas duplas.
\\	Insere uma barra invertida (*backslash*).
\a	Chamado de ASC bell ou *beep* do sistema. Se houver suporte, aciona um bipe.
\b	Aciona o *backspace*, ou seja, apaga o caractere anterior.
\f	Insere uma quebra de página.
\u	Insere um caractere UNICODE. Deve acompanhar um código com 4 números.

9. Estruturas de decisão

Veja a seguir os tipos de estruturas de decisão implementados em Python.

9.1. Operadores de teste

Operadores lógicos relacionais

Operador	Nome	Execução
==	Igual a	Verifica se um valor é igual ao outro
!=	Diferente de	Verifica se um valor é diferente ao outro
>	Maior que	Verifica se um valor é maior que outro
>=	Maior ou igual	Verifica se um valor é maior ou igual ao outro
<	Menor que	Verifica se um valor é menor que outro
<=	Menor ou igual	Verifica se um valor é menor ou igual ao outro

Operadores lógicos conectivos

Operador	Definição	Execução
and	E	Retorna True se ambas as afirmações forem verdadeiras
or	OU	Retorna True se uma das afirmações for verdadeira
not	NÃO	Retorna Falso se o resultado for verdadeiro

9.2. Estrutura de Decisão SIMPLES

A Estrutura de Decisão SIMPLES em Python deve obedecer à seguinte lógica de construção:

IF ( ): PRIMEIRO COMANDO ... ULTIMO COMANDO	O comando de teste IF deve conter os testes dentro de parënteses e finalizar com dois pontos (:). Os comandos ou instruções endentados que seguem são chamados de bloco. O bloco será executado apenas se o teste retornar VERDADEIRO. Caso o teste retorne falso, o bloco é ignorado. A primeira instrução não endentada depois do último comando marca o fim do bloco.

Não existe limite para o número de instruções dentro de um bloco de execução de uma Estrutura de Decisão Simples, que pode aparecer no corpo de uma instrução IF, mas tem que haver pelo menos uma.

Exemplos:

`if (x > 0):` `  print (‘x é positivo’)`	A expressão booleana depois da instrução if é chamada de condição. Se ela é verdadeira (true), então a instrução endentada é executada. Se não, nada acontece.

Exemplo de algoritmo/programa com a estrutura simples

```
# ========================================
'''
Seção de cabeçalho
Nome do algoritmo/programa: Lista estruturas de decisão -
Ex. 01
Função/Descrição: 1.    Escreva
um algoritmo que leia dois números inteiros.
Mostre na tela e diga se eles são iguais ou diferentes.
Autor: Waine Teixeira Júnior
Data: 31/05/2019
Número de controle de versão: 1.0
'''
# ========================================
'''
Seção de Declarações de variáveis

'''
# ========================================
# inicio do algoritmo/programa

#entrada de dados
print ('Digite o primeiro número')
n1 = int (input())
print ('Digite o segundo número')
n2 = int (input())

#processamento e saida

if (n1 == n2):
    print (n1, ' e ',n2,' são iguais')
if (n1 != n2):
    print (n1, ' e ',n2,' são diferentes')

# ========================================
# fim do algoritmo/programa
```

A seguir vamos conhecer os operadores lógicos de teste e operadores lógicos conectivos em Python.

9.3. Estrutura de Decisão COMPOSTA

A Estrutura de Decisão COMPOSTA em Python deve obedecer à seguinte lógica de construção:

IF ( ): COMANDO ELSE: COMANDO	O comando de teste IF deve conter os testes dentro de parënteses e finalizar com dois pontos (:). Da mesma forma que a estrutura simples, podem haver diversos COMANDOS dentro da estrutura, que serão executados quando o teste retornar VERDADEIRO. O que vai diferenciar é o ELSE. ELSE é uma instrução de teste, que manda executar outro bloco de comandos caso o teste do IF seja FALSO. A primeira instrução não endentada marca o fim do bloco.

Exemplo:

If (x % 2 == 0):
 	
print (x, ‘é par’)
 	
else:
 	
print (x, ‘é impar’)
	Se o resto da divisão de x por 2 for 0, então sabemos que x é par, e o programa exibe a mensagem para esta condição.
	
	Se a condição é falsa, o segundo grupo de instruções é executado. Desde que a condição deva ser verdadeira (True) ou falsa (False), precisamente uma das alternativas vai ser executada.

As alternativas são chamadas ramos (branches), porque existem ramificações no fluxo de execução.

Exemplo de algoritmo/programa com a estrutura composta

```
# ========================================
'''
Seção de cabeçalho
Nome do algoritmo/programa: Lista estruturas de decisão -
Ex. 01
Função/Descrição: 1.    Escreva
um algoritmo que leia dois números inteiros.
Mostre na tela e diga se eles são iguais ou diferentes.
Autor: Waine Teixeira Júnior
Data: 31/05/2019
Número de controle de versão: 1.0
'''
# ========================================
'''
Seção de Declarações de variáveis

'''
# ========================================
# inicio do algoritmo/programa

#entrada de dados
print ('Digite o primeiro número')
n1 = int (input())
print ('Digite o segundo número')
n2 = int (input())

#processamento e saida

if (n1 == n2):
    print (n1, ' e ',n2,' são iguais')
else:
    print (n1, ' e ',n2,' são diferentes')

# ========================================
# fim do algoritmo/programa
```

Vejamos a seguir como construir estruturas compostas encadeadas ou aninhadas.

9.4. Estruturas de decisão encadeadas

Nas estruturas de decisão encadeadas em Python, **uma estrutura de condição é aninhada dentro de outra**.

Portanto, teremos as mesmas possibilidades de encadeamento vistas em Portugol para o VISUALG.

a) Estruturas Condicionais Simples dentro de Estruturas Condicionais Simples

```
IF (    ): #aqui o início a estrutura de decisão simples
     COMANDO
                COMANDO
          IF (    ): #aqui o início a estrutura de decisão simples encadeada (interna)
```

b) Estruturas Condicionais Simples dentro de Estruturas Condicionais Compostas

```
IF (    ): #aqui o início da estrutura de decisão composta
     COMANDO
ELSE:
     IF (    ):  #aqui o início a estrutura de decisão simples encadeada (interna)
          COMANDO
```

c) Combinações entre Estruturas Condicionais Simples e Compostas

```
IF (    ): #aqui o início da estrutura de decisão composta
     COMANDO
ELSE:
     IF (    ):  #aqui o início a estrutura de decisão composta
encadeada (interna)
          COMANDO
     ELSE
          IF (    ): #aqui o início da estrutura dde decisão composta
encadeada (interna)
                COMANDO
          ELSE:
                IF (    ):  #aqui o início a estrutura de decisão simples encadeada (interna)
                    COMANDO
```

A seguir vamos estudar a estrutura de decisão de Múltipla Escolha.

9.5. Estrutura Múltipla Escolha

Não existe exatamente uma equivalência da estrutura ESCOLHA-CASO-OUTROCASO em Python. Todavia, o Python oferece uma estrutura bem semelhante para ser utilizada como equivalência: IF-ELIF-ELSE

A sintaxe do comando `if-elif-else` é a seguinte:

```
if ( ):
    # bloco contendo comandos a serem executados
elif ( ):
    # bloco contendo comandos a serem executados
elif ( ):
    # bloco contendo comandos a serem executados
# outros possíveis blocos de elif, cada um com a sua condição
else:
    # bloco contendo comandos a serem executados
comando_apos_if
```

Ou seja, o `if-elif-else` pode ter um ou mais blocos com `elif`, cada um com a sua condição específica.

Cada bloco (condição) é testado um de cada vez, até que uma condição seja satisfeita e apenas os comandos dentro desse bloco são executados.

Quando nenhuma condição é satisfeita, os comandos dentro do `else` são executados. Assim, **não** é necessário que haja um `else` no final.

Veja a seguir o código do nosso algorimto da calculadora:

```
print ("Digite a opção desejada")
print ("1 - para soma")
print ("2 - para subtração")
print ("3 - para multiplicação")
print ("4 - para divisão")
op = int(input())
if (op==1):
    print ("Você escolheu a soma.Digite o primeiro valor")
    n1 = int(input())
    print ("Digite o segundo valor a ser somado")
    n2 = int (input())
    soma = n1+n2
    print ("o valor da soma de ", n1, " por  ", n2, "  é ", soma)
elif (op==2):
    print ("Você escolheu a subtração. Digite o primeiro valor")
    n1 = int(input ())
    print ("Digite o segundo valor a ser subtraído")
    n2 = int(input ())
    subtracao = n1-n2
    print ("o valor da subtração de ", n1, " menos  ", n2, "  é ", subtracao)
elif (op==3):
    print ("Você escolheu a multiplicação. Digite o primeiro valor")
    n1 = int(input ())
    print ("Digite o segundo valor a ser multiplicado")
    n2 = int(input ())
    multiplicacao = n1*n2
    print ("o valor da multiplicação de ", n1, " vezes  ", n2, "  é ", multiplicacao)
elif (op==4):
    print ("Você escolheu a divisão. Digite o primeiro valor")
    n1 = int(input ())
    print ("Digite o segundo valor, que não pode ser zero")
    n2 = int(input ())
    if (n2==0):
        print ("não existe divisão por zero")
    else:
        divisao = n1/n2
        print ("o valor da soma de ", n1, " por  ", n2, "  é ", divisao)
else:
    print ("Opção inválida")
```

9.6. Comparação entre as estruturas de decisao

Dado o seguinte problema:

Elabore um algoritmo que verifque se um número lido pelo teclado é negativo, nulo ou positivo.

A) Solução com estrutura de decisão simples if:

```
numero = float(input("Digite um número: "))
if numero < 0:
   print("O número é negativo.")
if numero == 0:
   print("O número é nulo.")
if numero > 0:
   print("O número é positivo.")
```

B) Solução com estrutura de decisão composta if-else:

```
numero = float(input("Digite um número: "))
if numero < 0:
   print("O número é negativo.")
else:
   if numero == 0:
      print("O número é nulo.")
   else:
      print("O número é positivo.")
```

C) Solução com estrutura de decisão composta if-elif-else:

```
numero = float(input("Digite um número: "))

if numero < 0:
   print("O número é negativo.")
elif numero == 0:
   print("O número é nulo.")
else:
   print("O número é positivo.")
```

Quadro comparativo

numero = float(input("Digite um número: "))	numero = float(input("Digite um número: "))	numero = float(input("Digite um número: "))

if numero < 0:
print("O número é negativo.")
if numero == 0:
print("O número é nulo.")
if numero > 0:
print("O número é positivo.")
	if numero < 0:
	print("O número é negativo.")
	else:
	if numero == 0:
	print("O número é nulo.")
	else:
	print("O número é positivo.")
		if numero < 0:
		print("O número é negativo.")
		elif numero == 0:
		print("O número é nulo.")
		else:
		print("O número é positivo.")

3 testes nenhum encadeamento	2 testes um encadeamento	2 testes nenhum encadeamento